

# Лабораторная работа №2

Дисциплина: Научное программирование

Лабузов Александр Сергеевич

## Содержание

|                                      |   |
|--------------------------------------|---|
| Цель работы .....                    | 1 |
| Задание .....                        | 1 |
| Выполнение лабораторной работы ..... | 1 |
| Выводы.....                          | 7 |

## Цель работы

– Научиться оформлять отчёты с помощью легковесного языка разметки Markdown.

## Задание

- Сделать отчёт по предыдущей лабораторной работе в формате Markdown. – В качестве отчёта предоставить отчёты в 3 форматах: pdf, docx и md (в архиве, поскольку он должен содержать скриншоты, Makefile и т.д.)

## Выполнение лабораторной работы

- 1) Акканут на Github уже в наличии.
- 2) Настройка глобальных параметров Git.

```

alabu@DESKTOP-2C4QC80 MINGW64 ~
$ git config --global user.name "Labuzov Alex" git config --global user.email "xeal2k2@yandex.ru"
usage: git config [<options>]

Config file location
--[no-]global          use global config file
--[no-]system          use system config file
--[no-]local           use repository config file
--[no-]worktree        use per-worktree config file
-f, --[no-]file <file>
                        use given config file
--[no-]blob <blob-id>  read config from given blob object

Action
--[no-]get             get value: name [value-pattern]
--[no-]get-all        get all values: key [value-pattern]
--[no-]get-regexp      get values for regexp: name-regex [value-pattern]
--[no-]get-urlmatch    get value specific for the URL: section[.var] URL
--[no-]replace-all    replace all matching variables: name value [value-pattern]
--[no-]add             add a new variable: name value
--[no-]unset          remove a variable: name [value-pattern]
--[no-]unset-all      remove all matches: name [value-pattern]
--[no-]rename-section  rename section: old-name new-name
--[no-]remove-section  remove a section: name
-l, --[no-]list        list all
--[no-]fixed-value     use string equality when comparing values to 'value-pattern'
-e, --[no-]edit        open an editor
--[no-]get-color       find the color configured: slot [default]
--[no-]get-colorbool   find the color setting: slot [stdout-is-tty]

Type
-t, --[no-]type <type>
                        value is given this type
--bool                value is "true" or "false"
--int                 value is decimal number
--bool-or-int         value is --bool or --int
--bool-or-str         value is --bool or string
--path                value is a path (file or directory name)
--expiry-date         value is an expiry date

Other
-z, --[no-]null        terminate values with NUL byte
--[no-]name-only       show variable names only
--[no-]includes        respect include directives on lookup
--[no-]show-origin     show origin of config (file, standard input, blob, command line)
--[no-]show-scope      show scope of config (worktree, local, global, system, command)
--[no-]default <value>
                        with --get, use default value when missing entry
--[no-]comment <value>
                        human-readable comment string (# will be prepended as needed)

```

## Настройка

### 3) Генерация ssh ключа.

```

alabu@DESKTOP-2C4QC80 MINGW64 ~
$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/alabu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in
Your public key has been saved in .pub
The key fingerprint is:
SHA256:cTeMyo1cFGgwSa3+O+1KQ5K+mz0e5UdT+s20Z/xOVnw alabu@DESKTOP-2C4QC80
The key's randomart image is:
+--[ED25519 256]--+
|  .+ .O.         |
|  ..+.  O        |
|   O  O + +      |
|  ... + B . . .  |
|  O.. + S        E|
| . O.O . O O  .. |
| . +... ..O O    |
| =.O+.  O+O      |
| +O++OO .O.OO    |
+-----[SHA256]-----+

```

## Ssh ключ

### 4) Генерация рдр ключа. И базовая настройка: выбрал RSA and RSA и поставил длину ключа 4096 бит. Выбрал неограниченный срок действия.

```

alabu@DESKTOP-2C4QC80 MINGW64 ~
$ gpg --full-generate-key
gpg (GnuPG) 2.4.5-unknown; Copyright (C) 2024 g10 Code GmbH
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
  0 = key does not expire
  <n> = key expires in n days
  <n>w = key expires in n weeks
  <n>m = key expires in n months
  <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: xea1
Email address: xea12k2@yandex.ru
Comment: no
You selected this USER-ID:
  "xea1 (no) <xea12k2@yandex.ru>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/c/Users/alabu/.gnupg/openpgp-revocs.d/9F
8714EF7EC8F3B7AAEE78E9A00E2B43FC9655DE.rev'
public and secret key created and signed.

pub   rsa4096 2024-09-15 [SC]
      9FB714EF7EC8F3B7AAEE78E9A00E2B43FC9655DE
uid     xea1 (no) <xea12k2@yandex.ru>
sub   rsa4096 2024-09-15 [E]

```

*pgp* ключ

5) Ввод кодового слова.

Pinentry

×



Please re-enter this passphrase

Passphrase:

OK

Cancel

*keyword*

6) Нахождение идентификатора ключа.

```
alabuz@DESKTOP-2C4QC80 MINGW64 ~
$ gpg --list-secret-keys --keyid-format LONG
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 2 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 2u
[keyboard]
-----
sec   rsa4096/A00E2B43FC9655DE 2024-09-15 [SC]
      9FB714EF7EC8F3B7AAEE78E9A00E2B43FC9655DE
uid   [ultimate] xea1 (no) <xea12k2@yandex.ru>
ssb   rsa4096/D0606CC8B0808761 2024-09-15 [E]

sec   rsa4096/DF444229949DBE09 2024-09-14 [SC]
      E79547E009663036D0E298FEDF444229949DBE09
uid   [ultimate] Labuzov Alex (key) <xea12k2@yandex.ru>
ssb   rsa4096/6BDDC2B5026CDC9C 2024-09-14 [E]
```

## Идентификатор

7) Экспорт публичного ключа.

```

7abub@DESKTOP-2C4QCE8 MINGW64 ~
$ gpg --armor --export DF444229949D8E09
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGb18hABEAC/4pPCeoDfs6ZXdxZsy9tX+xis5WHrq1U3P4LBDhJLXAMPp93n
A9gb5139qMhG0D8jyQUYF5M9mb/WeYnU1A955boc6iGP0UJ8n3/np/XB54Unbt25Ro1
XJ2jHqOdKv9mzbjw4A0/INW+JCEzyCwYnBYg7CfGdCylVZk/JGzheSWhochddtLw
15900CA4tk2o/0NcFCoUhOCLgkCCJMMNZq57zJbWm2fTvpZM5RfdAx4ertfddN
i17BSqhn1nHnHkbb8BkZtthbTk4QJ1N23T3F17ACVnITGq40T2CSwT2MLnwXNv1n
5Tz2wCAbtDGDcKzWlNyGCPN8LnyUU+/yS1+2MQoq/ATfQfOmyht12Ml2Yht31nZc
3w9iCjexwPSyJ1SRR1yiQ5kUaUaqYbUrTR1RPVbX6hsFXCZGFK2bxbEsam42Rje
jFQ7v8LrMRJE84IuR8SwZrnf4g5yGOG7r17FanH4mivzi2E5uB85KnsA1+1SzP
zJh7JpH3mKqKuc3lbt0pVzAoO0FQx0wvWfK9j5fpEPFNk2mEWB89sJkYdAY81
03jhs+n3/6zfA24vhs6GoBglkXfTmsE1PA0L1F7UQ3xGo05x6MrnqcxRNLb9x7n2
HV9p5ZSSCO1HbWErq4Ld0FctVL9iUBB8LxoAHZ8e+xtH4LZrF0xFrql4LZrF0x7n
tCZMYWJ1em92IEFsZXggKGt1eSkqPHhIYwmyazJAeWfUzGV4LJn1PcKUCBQA0gA
y0hYBoeVrA+AJZjA200KY/t9EQimUnb4J8QJm5FIQAh5DBQsJCAcAiICBHQKQcL
AgQwAGMBAh4HAheAAAOJEN9EQimUnb4JdGfQAJ5R611TFnqN1i9zLLC0wJkvrkP
QAOQCTr01r1UtiEqpPx6D7JkQz+Rd8RkbgVaf+FaMrJrAn7S5s9F003zYp2FKa8v0
FQeGkW7F04UThdpf2QJgry+DvY7k1bl1r7nQ2BiI4BJJwnjTVxovWfwpPFJTwkNjD
eh8Ufai0XmPOZthZ6QyWyC80wbV2WzQ2BI//HE0AtJtTfChcEIDK9B1hwbA2MA
Bqj9qscf//KkfPvpxqWCUKUMY455bQDTSXh9x+9tJhSAh62SLYmq+Edoq42G7
FQhJbLQ3OKvac18XZddq56iZAxY3p4AmJhdd/JEBEBSqWx+Xy8fKpCqVem1t1iUtpB
7X0FVN00omCz44HNxoU2gRf42dGuJrFzdsxNPQ89d3VcpeRNbJpDJDSP2HB+8UO
ctxt9rAsCs8i1DDEsRrNhE0etFbZGrHm10439FV1S20w/vbVp8L2Yv23P006
6xJn2Z+9DpIyQFCHVFz3WEJTApzhEbGSLMG/e4hjvZGU01ZLSUCFzU4t4E4bVXL
/dxI5Y2Pch0b1U7Vmup4E2JkZTDFKPIB/LnKnPErmt+xlEgV9pFxuemuVM6D0mVx
ayY5QZrKfWepIhnp07F42g96+1VP1Z9cXAD+U1K5n8NmF7vRvpBMV4aP6/K92X
uEp5xtfFbkMtw4yQINBGb18hABEAC3rCfJscwLyrJkMqD7MLAmPkDYsgYIOXV
o8blyPT098Ay4dv4r6/3yuzcK420uU5csCwqNfzUwTerqNCiUd1eSRd+gTbAhJEuO
e1ldFVN2P0+EsSQZmozjZVfzYzXAKCOFb6CZYw0aqp1sJ0sHpn52gzdBeeZn5kH
m+2X1iV7ch0b18q1mEjLH5UoFmCaE0ar3p0q3vR6Ejg69LKnR6tLM/3kit/LV
qF4LNDMFmF4kK3ACc8NgZ3GL1Lud295JBDa62+jkEtiGmD0E51AyIvgbe2dQ8e
WqUJnE1t3QkDWMFXiZBi+4MEEm6766+ctZVZVOMBS5tAplGqAG9L70ppqCk0xfi
kTvGU0SL3DHUNU7RwyOPQ3PjgCwCv5yHbVkvG4j3eCnGjY09q9+tn/DNOV1d4C
Q0xKF4rP2p2Rbe9FA/d5wU1eUWstoz2xR/J1N7gj5TOar+Ival6WN0kOdm20Faw2
AoSmLPBTRi8cSDGHIzEqUY6FwmuZrfsN8bkRMEKf05GF/qswZ8oL9tbiLTHM30f
Zc1Dsmkj33h0uJ06bKKS8BMRKDTgMqvh13tPfJ6b4nKqGfQp6462Rhs9F3KODJNm
TF4JAag8U7j7Ne6bWOP9K0953emsTOovPqgdpysyzs1xeOaYn/q4dTrZ2k4Ny
vvn9A9uVoQARQA0ABiQI28BgBCAAQY1EE55VH4Al1mDbQ4j3+30RCZSdvdkFamb1
8hACGwAAGCgk30RCZS5dvgN1xw//fwYmg5p1VU1Pu1ztuPxbmrZ3ayDmlKs/jkX6J
jDFHxuR1ZSRs01TkTyvYo1iE/q1GAZQ75JHeuEYtEvY2ZIn9s8vSbc+y/u0NU
r40QxuzqGNbNK+64UUVvDT4Emmbd1B8iBix/w31Z2jppEKZ79xqQxxtr93b32pJuHt
g7H0SeadaKTr167qSnuws1yvzdYHs9T4i+r+kfPXB5XJ3xnpNFrNfyQQRFAcBqd4
3X5yqWU7X28Dv3DzvabMLX0/Ltyn8PqYU101YqYU+VY098FUB69uysQsR//2AL7/Ce
ne06vHRTa1319xpVl/x9hJpyVrWOT/n5FSXs6P0AvH7OLSyj82c2PrVNXwZqz
bdKofv9naPC/r6S5Wt0qygr9iufkXuH9RQEw1cuRb7d90CORY1LID/GU6AF4Yjyat
EedIcKavv880rMYLrkqhtat+zTnmCL8A8Fy/MI8A126mR661kPwk9JF68Q0ZMS
7EvXQfLO1QrKbHDLS6G6Uxeq9ZEhA0W9FBpna01BabGAKpheD1SoIy3jHD1LMczb
bcyLgk33LQr3c3LWymPjH4SAUT1AJmWbMLKQVFZ12d3xYV8GKM5AsVdLPu3aE
96nrY8bpzCQ7aB4Dv25H1SyJuvh3q+8VOyzZHebVUGOX85ig1z4Ny+jx/Rjey7n9
6k9VOBw=
=02+

-----END PGP PUBLIC KEY BLOCK-----

```

## Экспорт

Вторая часть закончена. Мы связали наш репозиторий с репозиторием на [github.com](https://github.com).

8) Конвертация отчёта из md с pandoc. ![конвертация] (image1/image7.PNG){width=70% }

## КОНТРОЛЬНЫЕ ВОПРОСЫ №2:

- 1) Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды `git` различными опциями. Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом.
- 2) В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять неполную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных. Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.
- 3) Централизованные системы — это системы, которые используют архитектуру клиент / сервер, где один или несколько клиентских узлов напрямую подключены к центральному серверу. Пример - Wikipedia. В децентрализованных системах каждый узел принимает свое собственное решение. Конечное поведение системы является совокупностью решений отдельных узлов. Пример — Bitcoin. В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером.
- 4) Создадим локальный репозиторий. Сначала сделаем предварительную конфигурацию, указав имя и email владельца репозитория: `git config --global user.name "Имя Фамилия"` `git config --global user.email "work@mail"` и настроив utf-8 в выводе сообщений `git config --global quotePath false` Для инициализации локального репозитория, расположенного, например, в каталоге `~/tutorial`, необходимо ввести в командной строке: `cd mkdir tutorial cd tutorial git init`
- 5) Для последующей идентификации пользователя на сервере репозитория необходимо сгенерировать пару ключей (приватный и открытый): `ssh-keygen -C "Имя Фамилия work@mail"` Ключи сохраняются в каталоге `~/.ssh/`. Скопировав из локальной консоли ключ в буфер обмена `cat ~/.ssh/id_rsa.pub | xclip -sel clip` вставляем ключ в появившееся на сайте поле.

- 6) У Git две основные задачи: первая — хранить информацию о всех изменениях в вашем коде, начиная с самой первой строчки, а вторая — обеспечение удобства командной работы над кодом.
- 7) Основные команды git: Наиболее часто используемые команды git: – создание основного дерева репозитория: `git init` – получение обновлений (изменений) текущего дерева из центрального репозитория: `git pull` – отправка всех произведённых изменений локального дерева в центральный репозиторий: `git push` – просмотр списка изменённых файлов в текущей директории: `git status` – просмотр текущих изменений: `git diff` – сохранение текущих изменений: – добавить все изменённые и/или созданные файлы и/или каталоги: `git add .` – добавить конкретные изменённые и/или созданные файлы и/или каталоги: `git add имена_файлов` – удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории): `git rm имена_файлов` – сохранение добавленных изменений: – сохранить все добавленные изменения и все изменённые файлы: `git commit -am 'Описание коммита'` – сохранить добавленные изменения с внесением комментария через встроенный редактор: `git commit` – создание новой ветки, базирующейся на текущей: `git checkout -b имя_ветки` – переключение на некоторую ветку: `git checkout имя_ветки` (при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой) – отправка изменений конкретной ветки в центральный репозиторий: `git push origin имя_ветки` – слияние ветки стекущим деревом: `git merge --no-ff имя_ветки` – удаление ветки: – удаление локальной уже слитой с основным деревом ветки: `git branch -d имя_ветки` – принудительное удаление локальной ветки: `git branch -D имя_ветки` – удаление ветки с центрального репозитория: `git push origin :имя_ветки`
- 8) Использование git при работе с локальными репозиториями (добавления текстового документа в локальный репозиторий): `git add hello.txt` `git commit -am 'Новый файл'`
- 9) Проблемы, которые решают ветки git: • нужно постоянно создавать архивы с рабочим кодом • сложно “переключаться” между архивами • сложно перетаскивать изменения между архивами • легко что-то напутать или потерять
- 10) Во время работы над проектом так или иначе могут создаваться файлы, которые не требуются добавлять в последствии в репозиторий. Например, временные файлы, создаваемые редакторами, или объектные файлы, создаваемые компиляторами. Можно прописать шаблоны игнорируемых при добавлении в репозиторий типов файлов в файл `.gitignore` с помощью сервисов. Для этого сначала нужно получить списки имеющихся шаблонов: `curl -L -s https://www.gitignore.io/api/list` Затем скачать шаблон, например, для C и C++ `curl -L -s https://www.gitignore.io/api/c >> .gitignore` `curl -L -s https://www.gitignore.io/api/c++ >> .gitignore`

## Выводы

Научиться оформлять отчёты с помощью легковесного языка разметки Markdown.  
Скомпилировал файлы из формата md в pdf и docx через pandoc.